Agentic AI Capstone: Five Full-Stack Projects

Architecture and build guide · Teams of six

16 October 2026

How to use this guide

Each of the five projects gives **every lecture from L4 to L16 a practical role**. You will build one system that brings together the layers covered in class: retrieval, tools over MCP, orchestration, memory, planning, a team of agents, an improvement loop, evaluation, human approval, security, and deployment.

L1–L3 underpin all of this. Every agent, critic, judge and reflection step in these projects uses a prompt. Write each prompt using L3's six slots (role, context, task, constraints, format, examples), and apply what you learned in L1–L2 about tokens and sampling when setting context length and temperature.

There is a lot to build, so the guide keeps the scope manageable.

Build the minimum first, then go deeper. Every project must meet the *minimum bar* on page 3: an honest, working end-to-end version of each idea. Each project then has **two signature challenges** that require deeper work and carry most of the technical marks. The cohort guideline still applies: it is better to develop two or three areas well than to cover ten superficially.

One architecture, five domains. All five projects use the reference architecture in Figure 1. Each brief adapts the diagram to its domain and uses numbered steps to follow one request from start to finish. Once you understand the reference diagram, you can follow each brief.

Already have an approved title? You can keep it. Use the reference architecture and minimum bar as a checklist, and use the brief closest to your domain as a model.

What this guide does not change. Dates, team rules and the marking scheme are set by the cohort's *Capstone Project Guidelines*: teams of six, with a final review that includes a presentation, demo and individual viva.

The reference architecture

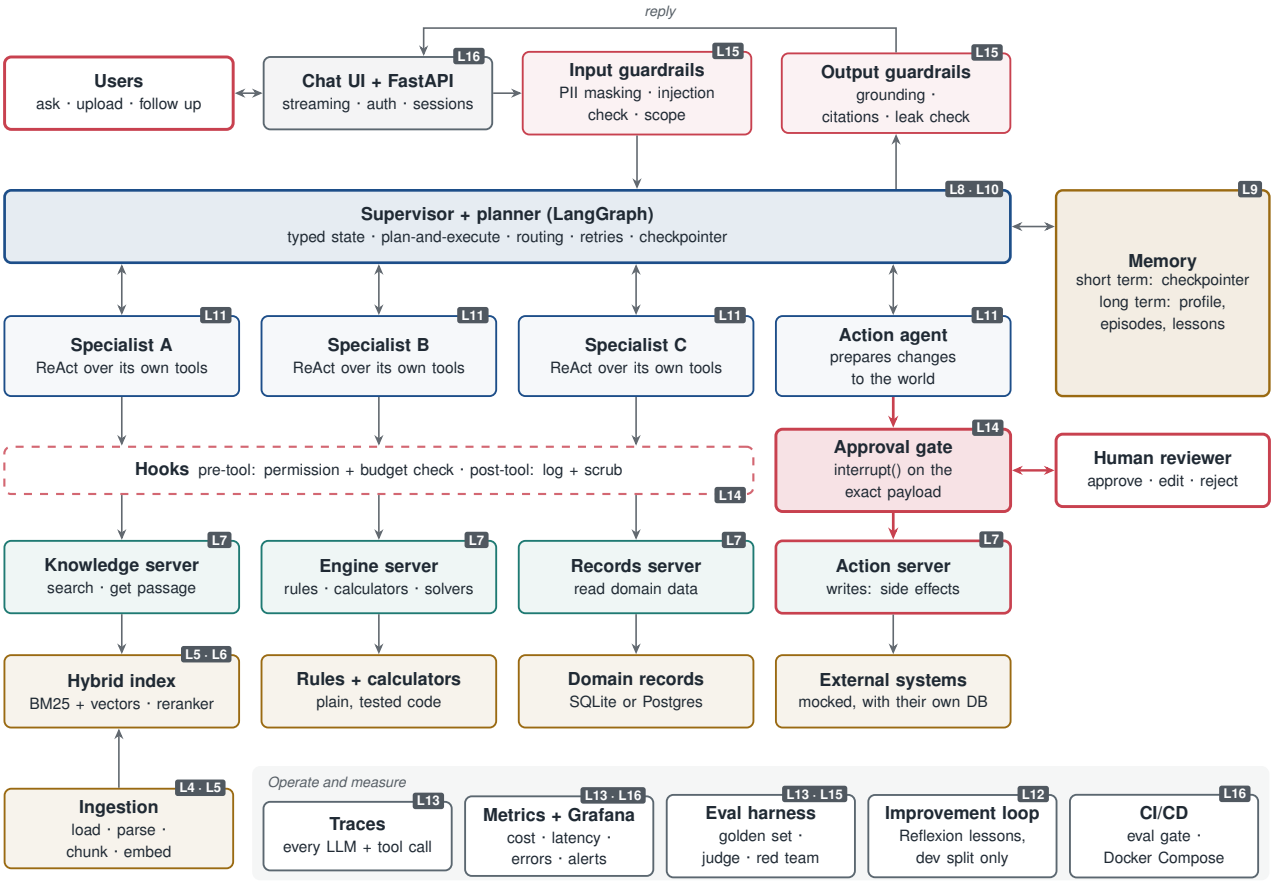


Figure 1: The reference architecture. Every box carries the lecture that teaches it. Coral marks control: guardrails, the approval gate and the human. The last agent and the last MCP server are the only ones that change the outside world, and they sit behind the approval gate.

Read the diagram from top to bottom. A request enters through the UI and **input guardrails**. The **supervisor** makes a plan and assigns parts of it to **specialist agents**. Each specialist uses its own tools in a ReAct loop. Every tool call passes through **hooks** that check permissions and budget before the call and log it afterwards. The tools run on **MCP servers** that provide access to knowledge, rules and records. An action that changes the outside world pauses at the **approval gate** until a person approves that exact action. The reply then passes through **output guardrails**. Traces, dashboards, the evaluation harness, the improvement loop and CI run alongside the whole process, as shown in the bottom band.

Three rules hold in every project:

1. **The model proposes; code and people decide.** Put arithmetic, eligibility rules, money and permissions in plain, tested code. The model selects the applicable rule and explains the result.
2. **Treat outside text as data, never as instructions.** Retrieved passages, tool outputs, emails and log lines may contain attacks.
3. **A person approves each exact change to the outside world.** Tie approval to the exact payload. Any subsequent change makes that approval invalid.

The minimum bar (every project)

Lecture	The minimum you must build	Evidence at the final review
L4–L6 Retrieval	Ingest data while retaining source and date metadata. Use hybrid search (BM25 and vectors combined with Reciprocal Rank Fusion), a reranker, and one query-side technique (rewriting, multi-query, HyDE or decomposition). Cite passage IDs in answers.	Recall@5 on 30+ labelled queries, measured before and after one change.
L7 Tools and MCP	Write at least three MCP servers with FastMCP. Use typed inputs and outputs, and return errors as structured results rather than crashes. Mark every tool as read or write, and give write access only to the agent that needs it.	A tool list with schemas and one test per tool.
L8 Orchestration	Build a LangGraph graph with typed state, conditional edges, one self-correction loop and a persistent checkpointer. Stop the process mid-run and resume it.	A graph diagram and a live demonstration of resume.
L9 Memory	Handle short-term context by trimming or summarising it. Add one long-term store and a written policy covering what is saved, when it is saved, who can read it, and how it can be corrected or deleted.	A one-page memory policy and a memory-on vs memory-off comparison.
L10 Planning	Show the user a plan object. Replan when a tool result contradicts it, and use ReAct within the specialists.	A trace showing a replan.
L11 Multi-agent	Use a supervisor and at least three specialists, each with its own tools and a typed hand-off message. Build a single-agent baseline with the same tools and budget.	A comparison of the single agent and the team.
L12 Improvement	Build a Reflexion loop. Run the agent on development cases and score each run with your checker code, using a judge where code cannot decide. Use a reflection step to turn each failure into a short written lesson that states when it applies. Have a person review every lesson before adding it to the playbook. Retrieve only lessons relevant to the case, and freeze the playbook before the held-out run. A DSPy optimiser is optional.	Held-out results before and after the playbook; the playbook; and one rejected lesson with the reason for rejecting it.
L13 Evaluation and monitoring	Build a golden set of 60+ cases split into development and held-out cases. Check outcomes, trajectories and tool use. Check an LLM judge from a different model family against 20+ human labels. Keep traces for every run and build a Grafana dashboard.	An evaluation report, judge agreement and a dashboard.
L14 Human control and hooks	Route every action that changes the world through `interrupt()` with approve, edit and reject options. Tie approval to the exact payload. Use pre-tool hooks to enforce permissions and budget, and post-tool hooks to log.	A live demonstration of an edit invalidating an earlier approval.
L15 Security	Write a one-page threat model. Add input and output guardrails, and give each agent only the tools it needs. Build a red-team suite of 30+ attacks, including injection through your own data path.	Attack success with defences off and on, and the rate at which defences block legitimate work.
L16 Deployment	Serve the app through FastAPI with streaming. Make `docker compose up` start the app, MCP servers, vector store, Prometheus and Grafana. Keep prompts and model IDs in one versioned config. Run evaluation in CI and fail below a threshold. Set up one working alert.	Installation on a clean machine and one failed CI run with an explanation.

Projects

No.	Project	Domain	Who uses it	Signature challenges
1	**Scheme Saathi**	Public services	Citizen and a CSC operator	Turning scheme rules written in prose into code; asking the fewest questions, in three languages
2	**ClaimSense**	Health insurance	Claims adjudicator	Clause-grounded money calculations; a playbook that learns from adjudicator corrections
3	**Socratic Tutor**	School education	Class 9–10 student, and their teacher	A learner model that drives a study plan; measuring teaching with simulated students
4	**On-Call Copilot**	IT operations	On-call engineer	Diagnosis as planning over live telemetry; safe, reversible actions
5	**Analyst Desk**	Finance	Equity research analyst	Numbers you can trace to a filing; parallel research with a critic

Projects 1 and 2 focus on decisions grounded in rules and documents. Project 3 focuses on memory and planning; Project 4, on safe action in a live system; and Project 5, on research orchestration and numerical accuracy. The table below shows how each project covers every lecture. **Bold** marks its signature areas.

How each project covers L4–L16

	1 Scheme Saathi	2 ClaimSense	3 Socratic Tutor	4 On-Call Copilot	5 Analyst Desk
L4–L6 Retrieval	Scheme clauses, queries in three languages	Policy clauses per bill line	NCERT sections and exemplars	Runbooks and postmortems	10-K sections and tables
L7 MCP tools	**Rule engine**, document check, portal	**Calculator**, tariff, claims DB	Solver, learner store, question bank	Telemetry, allow-listed actions	EDGAR, sandbox, portal
L8 Orchestration	Plan, rules, draft; resumable	Line-item pipeline with critic loop	Practice and quiz modes	Incident state machine	Fan-out with critic loop
L9 Memory	Household profile with consent	Claim history, sub-limits used	**Learner model**	Postmortem lessons	Past notes, preferences
L10 Planning	**Question planning**	Admissibility, then adjudication	Study plan over a concept graph	**Hypothesis testing, replanning**	Research outline
L11 Multi-agent	Intake, eligibility, documents, filing	Extract, cover, fraud, critic	Diagnose, tutor, grade, set	Parallel diagnosis	**Parallel researchers, critic**
L12 Improvement	Lessons from extraction mistakes	**Playbook from adjudicator edits**	Teaching lessons per misconception	Lessons from development incidents	Lessons from critic rejections
L13 Evaluation	Personas, simulated citizen	Hand-made worksheets	**Simulated students**	Injected faults give exact truth	**XBRL gives exact truth**
L14 Human control	Citizen confirms exact form	Two-level approval	Teacher signs question sets	**Approve exact command**	Edit plan, sign note
L15 Security	Personal data, scam pages, fraud requests	Injected notes, forged bills	Minors, answer extraction	Injection through logs	Injected web text, private watchlist
L16 Operations	Chat service plus portal	Turnaround dashboard	Teacher dashboard	An agent inside an observed system	Cost per note

Team roles

The six roles correspond to parts of the architecture. Assign names to them at the start and keep a contribution log from the first day.

1. **Domain and product lead.** Defines the problem, users, data, written policies and golden-set content. Talks to the domain expert and owns the demo script.
2. **Knowledge and memory engineer.** Handles ingestion, retrieval, reranking, the memory store and its policy (L4–L6, L9).
3. **Agents and orchestration engineer.** Builds the LangGraph graph, planner, specialists and improvement loop (L8, L10–L12).
4. **Tools, control and security engineer.** Builds the MCP servers, hooks, approval gate and guardrails, and writes the threat model (L7, L14, L15).

5. **Evaluation and red-team engineer.** Builds the evaluation harness and handles judge calibration, the held-out cases, the red-team suite and the experiment results (L13, L15).
6. **Deployment and operations engineer.** Builds the FastAPI service and UI, Docker Compose, CI with the evaluation gate, tracing, dashboards and alerts (L13, L16).

Two separations keep the results honest. The person tuning the agent must not write the held-out cases, and the person building the defences must not write the attacks. Role 5 owns both; role 1 checks the held-out cases against the written policy. In the viva, everyone should be able to redraw the full architecture from memory and explain their own part in depth.

Common stack

Use the tools introduced in the labs and HW2. Use plain Python wherever it makes the design clearer.

Layer	Default	Notes
Models	Use OpenRouter. For specialists, use `openai/gpt-4o-mini` or `openai/gpt-4.1-mini`; use a stronger model for the supervisor and planner.	In our L11 lab, `gpt-4o-mini` was unreliable as a coordinator, while `gpt-4.1-mini` worked. Keep the model fixed when comparing designs and record the exact model IDs.
Judge	Use a model from a different family, such as `google/gemini-3.1-flash-lite` or `meta-llama/llama-3.3-70b-instruct`.	Check its decisions against human labels before relying on it.
Orchestration	Use LangGraph, a SQLite or Postgres checkpointer, and a LangGraph store for long-term memory.	Use `interrupt()` and `Command(resume=...)` for approvals.
Tools	Use FastMCP servers and `langchain-mcp-adapters` to load them into LangGraph.	Use stdio during development and HTTP inside Docker.
Retrieval	Use Chroma or Qdrant, `rank-bm25`, and a cross-encoder reranker.	For embeddings, use local MiniLM or bge on CPU, or OpenRouter embedding models (it lists `baai/bge-m3` and `intfloat/multilingual-e5-large`, both multilingual).
Guardrails	Use Llama Guard 4 (`meta-llama/llama-guard-4-12b` on OpenRouter), a small injection classifier such as Llama Prompt Guard 2 on CPU, and your own rules in hooks.	Use several layers of guardrails rather than a single filter.
Improvement	Write a Reflexion loop with an agent, evaluator and reflection step, and keep its lessons in the LangGraph store.	Build and review lessons only on development cases. DSPy is optional.
Evaluation	Use pytest, a JSONL runner, your checker code, and RAGAS for retrieval metrics if useful.	Keep gold answers out of prompts and tools.
Observability	Use LangSmith or self-hosted Langfuse for traces, with `prometheus_client` in FastAPI feeding Prometheus and Grafana.	Record cost, latency, errors and outcomes for each run.
Serving	Use FastAPI with streaming, Streamlit, Chainlit or Gradio for the UI, Docker Compose, and GitHub Actions.	No GPU is needed.

Evaluation protocol (every project)

Golden set. Prepare at least 60 cases, and write each expected outcome *before* running the agent on that case. Keep related variants of a scenario together when splitting the set into about 20 development cases and at least 40 held-out cases. Store held-out cases in a folder the agent cannot read.

Check with code first. Use code to check the final database state, amounts, the existence of cited passages, tool calls and forbidden actions. Use the judge for aspects code cannot check, such as explanation quality or whether a passage supports a claim. Calibrate the judge on 20+ outputs you labelled by hand and report agreement using Cohen's kappa.

Four experiments, one harness. Represent each experiment with a config file rather than a separate code

branch.

- **E1, does the agent justify its cost?** Compare C0, a fixed workflow or single retrieval prompt with no agent decisions; C1, one agent with all the tools; and C2, your team of agents. Use the same model, tools and budget.
- **E2, do the defences work?** Run the system with defences off and on, both on the red-team suite to measure attack success and on clean cases to measure how often legitimate work is blocked.
- **E3, did the lessons help?** Compare held-out results before and after adding the playbook. Include the cost of generating lessons, and identify any lesson that made results worse.
- **E4, the signature ablation.** Use the ablation specified in the project brief.

Report the results accurately. For every configuration, give task-success counts, policy violations, attack success, false blocks, cost per case, median and 95th-percentile latency, and errors. Include failed and timed-out runs. Repeat the final held-out comparison three times to account for variation in LLM outputs, and report the spread. If you make a change after viewing held-out results, treat those cases as development cases and state that you did so.

What to submit

Submit a repository with a README that starts everything with one command. Include your own redrawn architecture diagram and numbered walkthrough, a threat model page, a memory policy page, an evaluation report (4–6 pages), an experiment log, a contribution log, slides, and a backup demo recording. Do not include API keys, private data or material you cannot redistribute.

The cohort rubric determines the marks, with roughly equal weight for scope and impact, technical depth and execution, presentation, and individual viva. Provide the following evidence for each part:

- **Scope and impact:** identify a real user, measure an outcome, and state the limits accurately.
- **Technical depth:** meet the minimum bar throughout, complete both signature challenges properly, and run fair experiments.
- **Presentation:** trace one request through the architecture and present numbers with context.
- **Viva:** each person explains their part and can redraw the architecture and justify every box.

5. Analyst Desk: research notes where every number checks out

In one line. Build a team of research agents that reads company filings, calculates numbers in code, compares years and peers, and drafts a cited note for an analyst to sign before publication.

Who it is for. An equity research analyst at an asset manager. Preparing a company note involves reading annual reports (10-K), extracting numbers, calculating ratios, comparing the company with peers and its previous year, and tracking changes in risk factors. A single wrong number can undermine trust in the entire note.

Why US filings. SEC EDGAR is free. Its XBRL APIs return reported numbers as JSON without an API key, providing exact ground truth for each figure. EDGAR asks automated users to identify themselves through a User-Agent with contact details and remain below 10 requests per second, so cache everything locally. For an external check, the FinanceBench open sample contains 150 questions about public filings with evidence (licence on Hugging Face: CC BY-NC 4.0). You can use Indian annual reports, but you would need to label the numbers by hand.

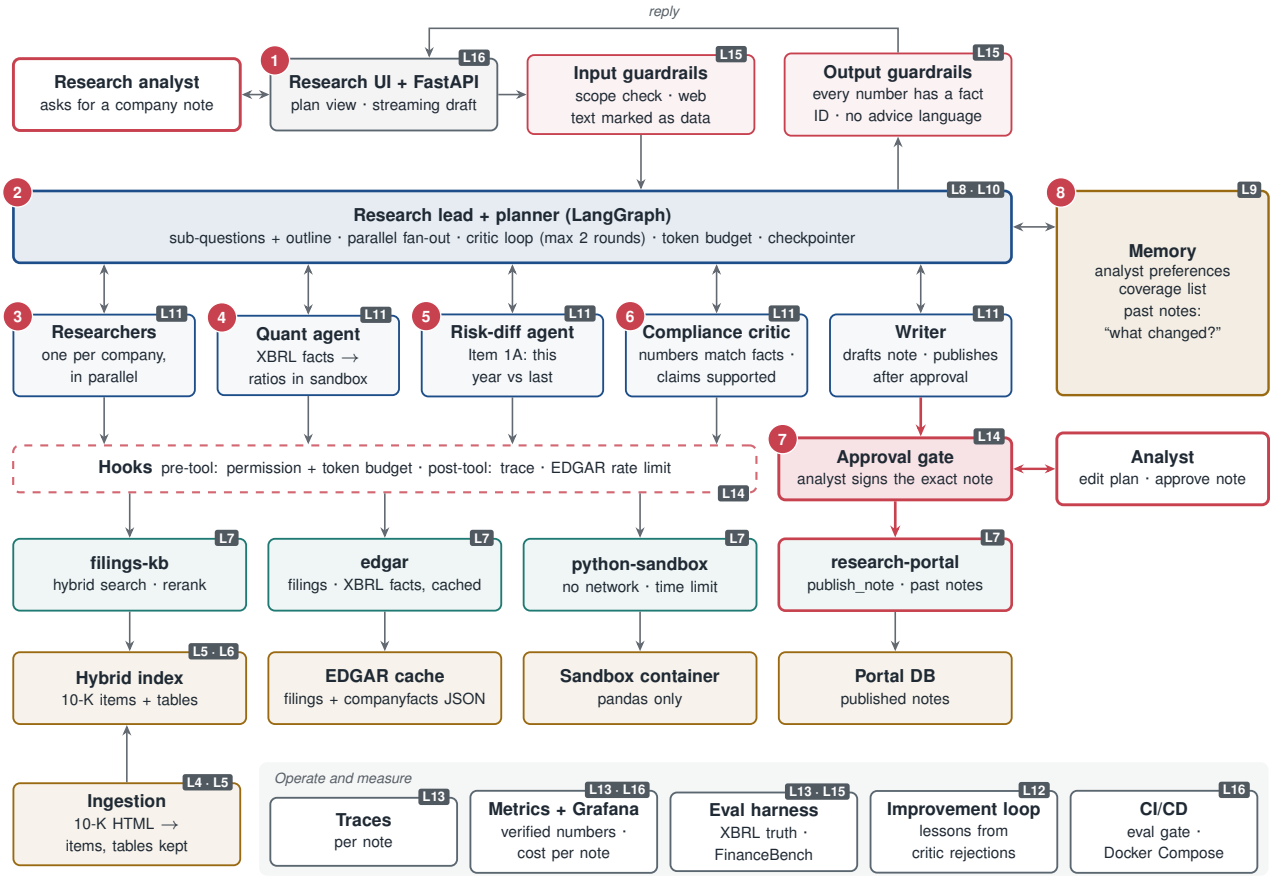


Figure 6: Analyst Desk. Numbers match the walkthrough below.

One request, start to finish. An analyst asks: *"Compare Microsoft's and Alphabet's operating margins over the last three fiscal years, and tell me what changed in their risk factors."*

1. The analyst enters the request in the **research UI**. Input guardrails check its scope and treat any web text as data.
2. The **research lead** writes sub-questions and an outline. The analyst can edit the plan through an *edit* interrupt. The note receives a token budget.
3. **Filings researchers** work in parallel, one per company. They use hybrid search and reranking over 10-K sections (Item 7 MD&A, Item 1A Risk Factors) and return findings with passage IDs.
4. The **quant agent** retrieves XBRL facts (revenue, operating income) and calculates margins in the sandbox. Each number has a fact ID containing its concept, period and filing.
5. The **risk-diff agent** compares this year's Item 1A with last year's and cites risks that were added, removed or materially changed.
6. The writer drafts the note. The **compliance critic** checks each number against its fact using code, each claim against its passage using a judge, and the wording against rules that prohibit promises of returns. It

can return the draft for at most two revisions.
7. The **analyst approves the exact note**. Only then does `publish_note` write it to the research portal.
8. **Memory** stores the note in the coverage history, allowing questions such as *"What changed since our last note?"* It also records preferences such as *"always include a peer table"*.

Signature challenges.

- **A. Numbers you can trust.** Link every number in the note to an XBRL fact or cited table cell, and check it with code. Fiscal years differ (Microsoft's ends in June, Alphabet's in December), as do units, so make like-for-like comparisons.
- **B. Deep-research orchestration.** Use parallel fan-out, a critic loop and a token budget. Compare the result with a single agent given the same tools and budget, and report quality, cost and time.

What you build.

Layer	Build this
Retrieval (L4–L6)	Cover 10–15 companies in two sectors over three years each. Split 10-K HTML by Item, keep tables as tables, and attach company, year and item metadata. Use hybrid search, a reranker and query decomposition.
Tools (L7)	Build `filings-kb`, `edgar` (list filings, get section, XBRL fact; cached), `python-sandbox` (no network, time limit), and `research-portal` (past notes; `publish_note` behind the gate).
Orchestration and planning (L8, L10)	Build a plan and outline, parallel branches, a critic loop of at most two rounds, and a budget enforced by hooks.
Memory (L9)	Store analyst preferences, the coverage list and past notes. Keep the private watchlist within the system.
Agents (L11)	Use filings researchers, quant, risk-diff, compliance critic, writer and publisher agents.
Improvement (L12)	Use Reflexion to turn critic rejections and analyst edits on development questions into research lessons (for example, *"align fiscal years before comparing companies"*). Review and freeze them before the held-out run.
Human control (L14)	Let the analyst edit the plan and sign the exact note.
Security (L15)	Test an injected "news article" that tries to make the note recommend a stock, attempts to put the private watchlist in a search query, and code in the sandbox trying to reach the network.
Evaluation (L13)	Prepare 60 research questions, including numeric questions checked automatically against XBRL, comparisons, and risk-change questions labelled by hand for 10 company-years. Use FinanceBench's open sample as an external retrieval check.
Operations (L16)	Show the share of numbers verified, critic rejections, tokens and cost per note, EDGAR cache hits and latency on a dashboard. Alert when the verified share falls below 98%.

Signature ablation (E4). Compare the critic loop on and off, and table-aware parsing with plain-text chunks.

Demo. (1) Show the margin comparison with every number linked to its fact. (2) Show the critic finding and correcting a planted wrong number, while the injected article's "buy" recommendation stays out of the note. (3) Answer *"What changed since our last note?"* using memory.

Watch out for. Do not calculate numbers inside the LLM, mix fiscal and calendar years or thousands and millions, or call EDGAR without a cache.

Questions every team should be ready for

1. Trace one request through your architecture diagram. Which boxes did it pass through, and what did each decide?
2. Which decisions were made by the model, and which by code, rules or a person?
3. Show your single-agent baseline. What benefit did the team of agents provide, and at what cost?
4. Which experiment led you to change the design?
5. Show the traces for how the system handled missing evidence, a failed tool and an injected instruction.
6. What does your memory store retain, why does it retain it, and how can it be deleted?
7. What lessons did the improvement loop learn, and how do you know they help on unseen cases?
8. What would be required before a supervised pilot with real users?

Record decisions and contributions from the first day. AI coding tools may write parts of the implementation. You must do the problem framing, design, evaluation and interpretation yourselves.